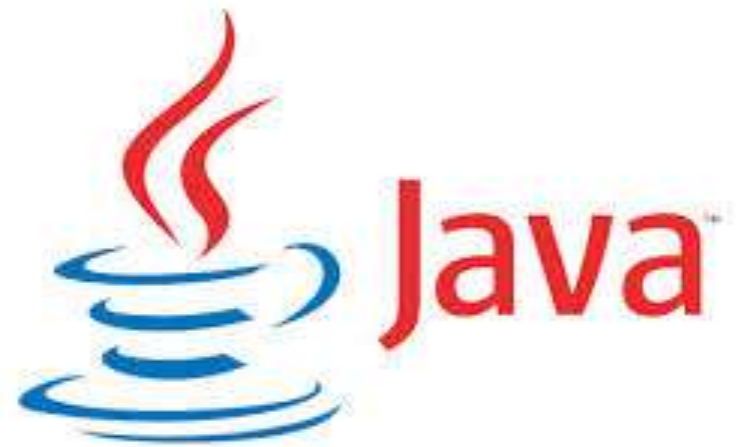




SOC2030

Application Programming in Java



Week 2

Dr. Andrei Dragunov

Introduction to Classes, Objects, Methods and Strings

- Java Classes & Objects
- Object Instantiation
- Instance Variables
- Class Variables
- Constructors
- Instance Methods
- Class Methods
- The this keyword
- Passing and returning objects
- Static Methods, static Fields
- Declaring Methods with Multiple Parameters
- Passing Objects to Methods
- Scope of Declarations

Some Common Paradigms

- **Imperative**: Programming with an explicit sequence of commands that update state.
- **Declarative**: Programming by specifying the result you want, not how to get it.
- **Structured**: Programming with clean, goto-free, nested control structures.
- **Procedural**: Imperative programming with procedure calls.
- **Functional**: Programming with function calls that avoid any global state.
- **Object-Oriented**: Programming by defining objects that send messages to each other. Objects have their own internal (encapsulated) state and public interfaces.
- **Logic** (Rule-based): Programming by specifying a set of facts and rules. An engine infers the answers to questions.

Java OOPs Concepts

Def. **Object-oriented programming (OOP)** is a programming paradigm based on the concept of "objects", which may contain data, in the form of fields, often known as attributes; and code, in the form of procedures, often known as methods.

Simula is considered as the first object-oriented programming language. The programming paradigm where everything is represented as an object, is known as truly object-oriented programming language.

- **Smalltalk** is considered as the first truly object-oriented programming language.

Object Oriented Programming

Object means a real world entity such as pen, chair, table etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Advantage of OOPs over Procedure-oriented programming language

- OOPs makes development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grows.
- OOPs provides data hiding whereas in Procedure-oriented programming language a global data can be accessed from anywhere
- OOPs provides ability to simulate real-world event much more effectively. We can provide the solution of real word problem if we are using the Object-Oriented Programming language

Naming conventions in java

Name	Convention
class name	should start with uppercase letter and be a noun e.g. String, Color, Button, System, Thread etc.
interface name	should start with uppercase letter and be an adjective e.g. Runnable, Remote, ActionListener etc.
method name	should start with lowercase letter and be a verb e.g. actionPerformed(), main(), print(), println() etc.
variable name	should start with lowercase letter e.g. firstName, orderNumber etc.
package name	should be in lowercase letter e.g. java, lang, sql, util etc.
constants name	should be in uppercase letter. e.g. RED, YELLOW, MAX_PRIORITY etc.

Object in Java

An entity that has state and behavior is known as an object e.g. chair, bike, marker, pen, table, car etc. It can be physical or logical (tangible and intangible). The example of intangible object is banking system.

An object has three characteristics:

- **state:** represents data (value) of an object.
- **behavior:** represents the behavior (functionality) of an object such as deposit, withdraw etc.
- **identity:** Object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. But, it is used internally by the JVM to identify each object uniquely.

Class in Java

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:

- **fields**
- **methods**
- **constructors**
- **blocks**
- **nested class and interface**

- **Instance variable in Java**

A variable which is created inside the class but outside the method, is known as instance variable. Instance variable doesn't get memory at compile time. It gets memory at run time when object(instance) is created. That is why, it is known as instance variable.

- **Method in Java**

In java, a method is like function i.e. used to expose behavior of an object

(Advantage: Code Reusability, Code Optimization)

3 Ways to initialize object

There are 3 ways to initialize object in java.

- **By reference variable**
- **By method**
- **By constructor**

Reference Variables and Reference Types

- Objects are accessed via the object's reference variables, which contain references to the objects. Such variables are declared using the following syntax:

ClassName objectRefVar;

A class is essentially a programmer-defined type. A class is a reference type, which means that a variable of the class type can reference an instance of the class. The following statement declares the variable `myCircle` to be of the `Circle` type:

Circle myCircle;

The variable `myCircle` can reference a `Circle` object. The next statement creates an object and assigns its reference to `myCircle`:

myCircle = new Circle();

Accessing an Object's Data and Methods

In OOP terminology, an object's member refers to its data fields and methods. After an object is created, its data can be accessed and its methods can be invoked using the *dot operator* (`.`), also known as the *object member access operator*:

- **`objectRefVar.dataField`** references a data field in the object.
- **`objectRefVar.method(arguments)`** invokes a method on the object.

Reference Data Fields and the **null** Value

The data fields can be of reference types. For example, the following **Student** class contains a data field **name** of the **String** type. **String** is a predefined Java class.

```
class Student {  
    String name; // name has the default value null  
    int age; // age has the default value 0  
    boolean isScienceMajor; // isScienceMajor has default  
value false  
    char gender; // gender has default value '\u0000'  
}
```

If a data field of a reference type does not reference any object, the data field holds a special Java value, **null**. **null** is a literal just like **true** and **false**. While **true** and **false** are Boolean literals, **null** is a literal for a reference type. The default value of a data field is **null** for a reference type, **0** for a numeric type, **false** for a **boolean** type, and **\u0000** for a **char** type. However, Java assigns no default value to a local variable inside a method.

Initialization through reference variable

- Initializing object simply means storing data into object. Let's see a simple example where we are going to initialize object through reference variable.

```
class B{
    int id;
    String name;
}

public class A {

    public static void main(String[] args) {
        B newObject;
        newObject = new B();
        newObject.name = "Name for B";
        System.out.println(newObject.name);
    }
}
```


The **this** Reference

- *The keyword **this** refers to the object itself. It can also be used inside a constructor to invoke another constructor of the same class.*
- The **this** keyword is the name of a reference that an object can use to refer to itself. You can use the **this** keyword to reference the object's instance members.

The **this** keyword can be used to reference a class's *hidden data fields*.

The **this** keyword can be used to invoke an overloaded constructor

Initialization through method

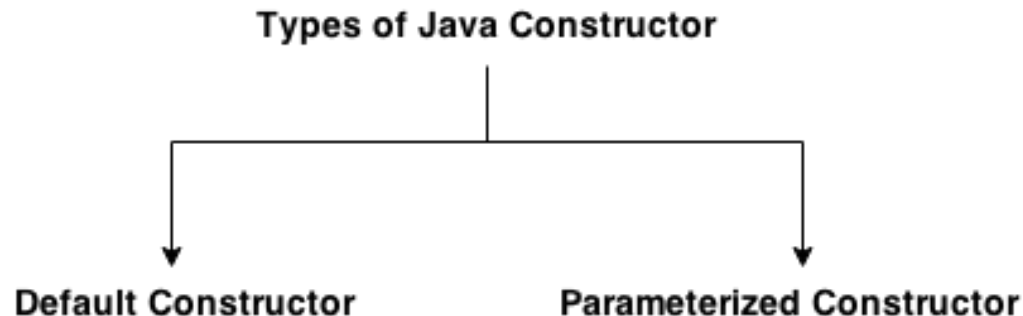
```
class B{
    int id;
    String name;
    void addvalues(int id, String name){
        this.id = id;
        this.name = name;
    }
}

public class A {
    public static void main(String[] args) {
        B newObject;
        newObject = new B();
        newObject.addvalues(1, "Name for B");
        System.out.println(newObject.id + " " + newObject.name);
    }
}
```

Class Constructor

- Constructors are a special kind of method. They have three peculiarities:
- A constructor must have the same name as the class itself.
- Constructors do not have a return type—not even **void**.
- Constructors are invoked using the **new** operator when an object is created.

Constructors play the role of initializing objects.



The constructor has exactly the same name as its defining class.

- Like regular methods, constructors can be overloaded (i.e., multiple constructors can have the same name but different signatures), making it easy to construct objects with different initial data values.
- A class normally provides a constructor without arguments (e.g., **Circle()**). Such a constructor is referred to as a *no-arg* or *no-argument constructor*.
- A class may be defined without constructors. In this case, a public no-arg constructor with an empty body is implicitly defined in the class. This constructor, called a *default constructor*, is provided automatically *only if no constructors are explicitly defined in the class*.

```
class B{
    int id;
    String name;
    B(int id, String name){
        this.id = id;
        this.name = name;
    }
    void addvalues(int id, String name){
        this.id = id;
        this.name = name;
    }
}

public class A {
    public static void main(String[] args) {
        B newObject;
        newObject = new B(0, "Unknown");
        System.out.println(newObject.id + " " + newObject.name);
        newObject.addvalues(1, "Name for B");
        System.out.println(newObject.id + " " + newObject.name);
    }
}
```

Using **this** to Invoke a Constructor

```
public class Circle {  
    private double radius;
```

```
    public Circle(double radius) {  
        this.radius = radius;  
    }
```

The **this** keyword is used to reference the hidden data field `radius` of the object being constructed.

```
    public Circle() {  
        this(1.0);  
    }
```

The **this** keyword is used to invoke another constructor.

```
    ...  
}
```

Constructor Overloading in Java

- Constructor overloading is a technique in Java in which a class can have any number of constructors that differ in parameter lists. The compiler differentiates these constructors by taking into account the number of parameters in the list and their type.

Difference between constructor and method in java

Java Constructor

Constructor is used to initialize the state of an object.

Constructor must not have return type.

Constructor is invoked implicitly.

The java compiler provides a default constructor if you don't have any constructor.

Constructor name must be same as the class name.

Java Method

Method is used to expose behaviour of an object.

Method must have return type.

Method is invoked explicitly.

Method is not provided by compiler in any case.

Method name may or may not be same as class name.

Anonymous object

Anonymous simply means nameless. An object which has no reference is known as anonymous object. It can be used at the time of object creation only.

If you have to use an object only once, anonymous object is a good approach.

```
class B{
    int id;
    String name;
    B(int id, String name){
        this.id = id;
        this.name = name;
        System.out.println("Object of class B been created;");
    }

    void printSomething(){
        System.out.println("Hello world");
    }
}

public class A {
    public static void main(String[] args) {
        new B(2, "test").printSomething();
    }
}
```

Visibility Modifiers

- *Visibility modifiers can be used to specify the visibility of a class and its members.*

You can use the **public** visibility modifier for classes, methods, and data fields to denote that they can be accessed from any other classes.

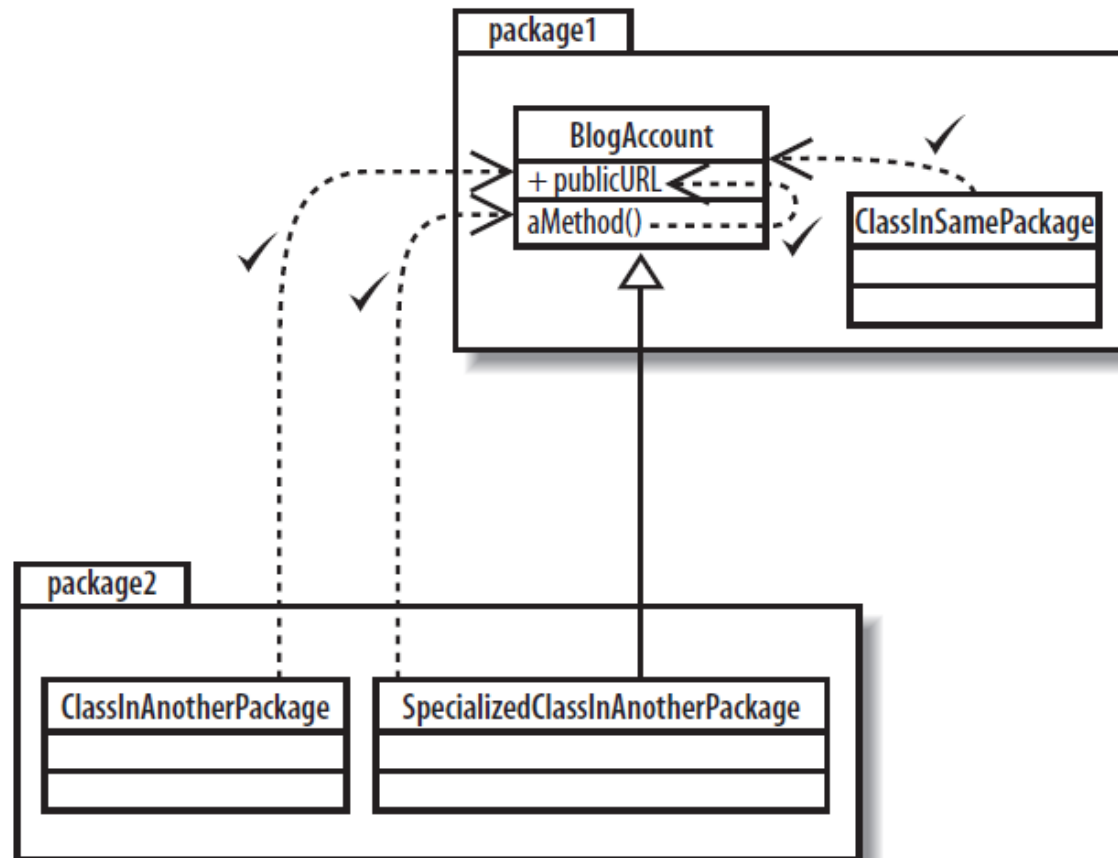
If no visibility modifier is used, then by default the classes, methods, and data fields are accessible by any class in the same package. This is known as *package-private* or *package-access*.

- In addition to the **public** and default visibility modifiers, Java provides the **private** and **protected** modifiers for class members. This section introduces the **private** modifier.
- The **private** modifier makes methods and data fields accessible only from within its own class.

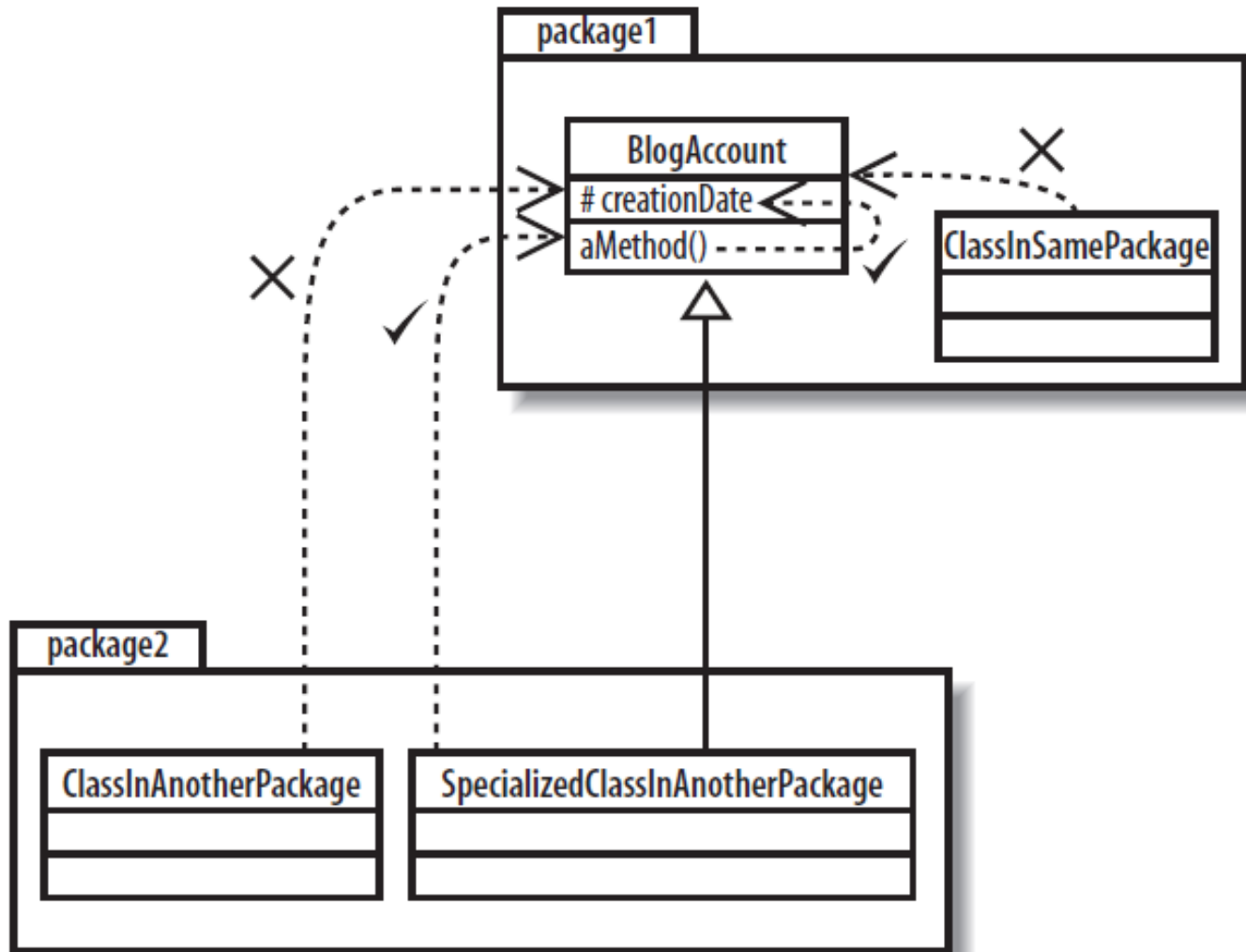
If you specify this keyword before the definition of a method, instance variable or class variable	Then code in the following places can call the method, or read and write the variable
public	Any code anywhere.
protected	Only code in the same package as this class as well as code in any subclasses, even if they are not in the same package.
(nothing)	Only code in the same package as this class. This is the default.
private	Only code in this class.

Public Visibility

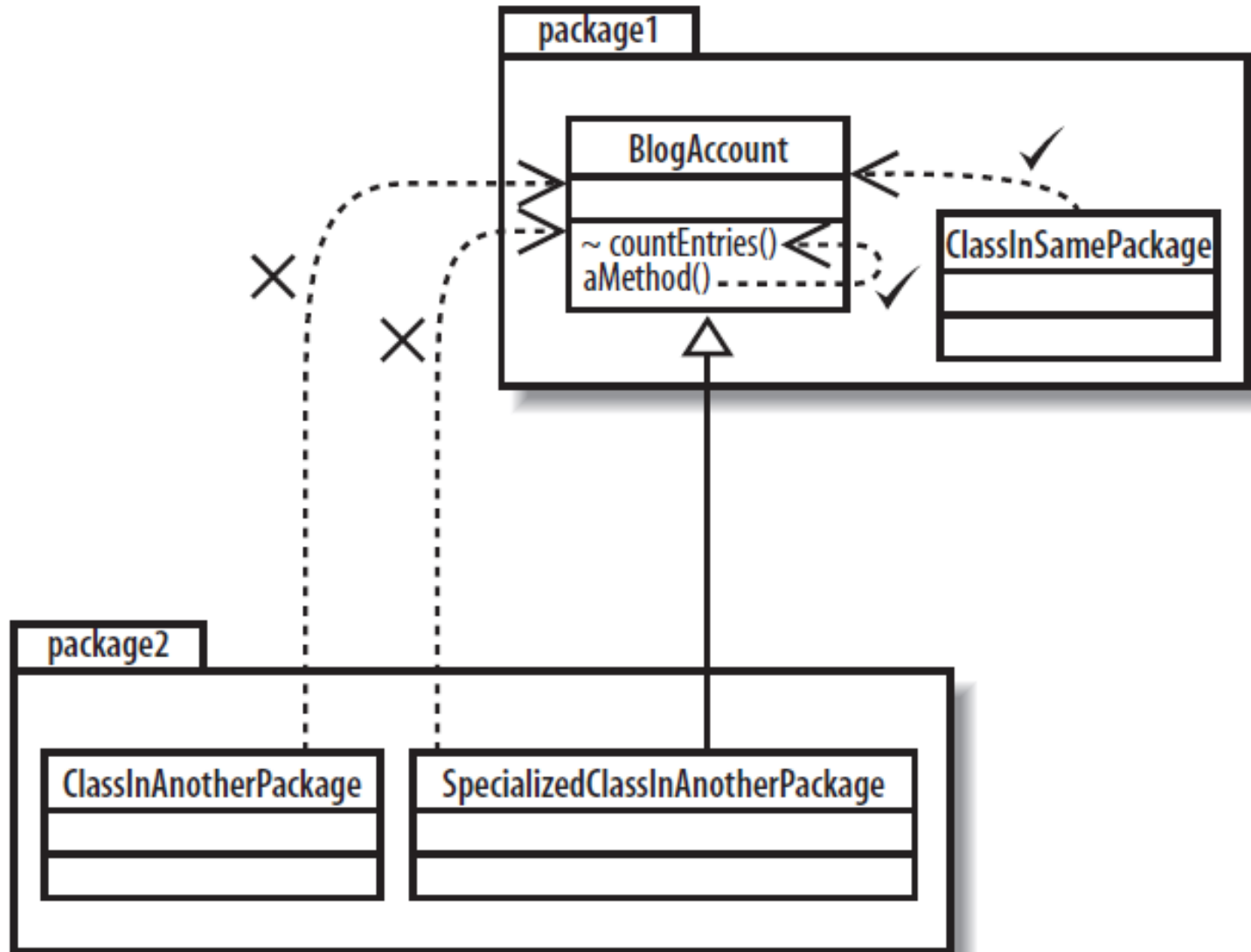
- Starting with the most accessible of visibility characteristics, public visibility is specified using the plus (+) symbol before the associated attribute or operation. Declare an attribute or operation public if you want it to be accessible directly by any other class.



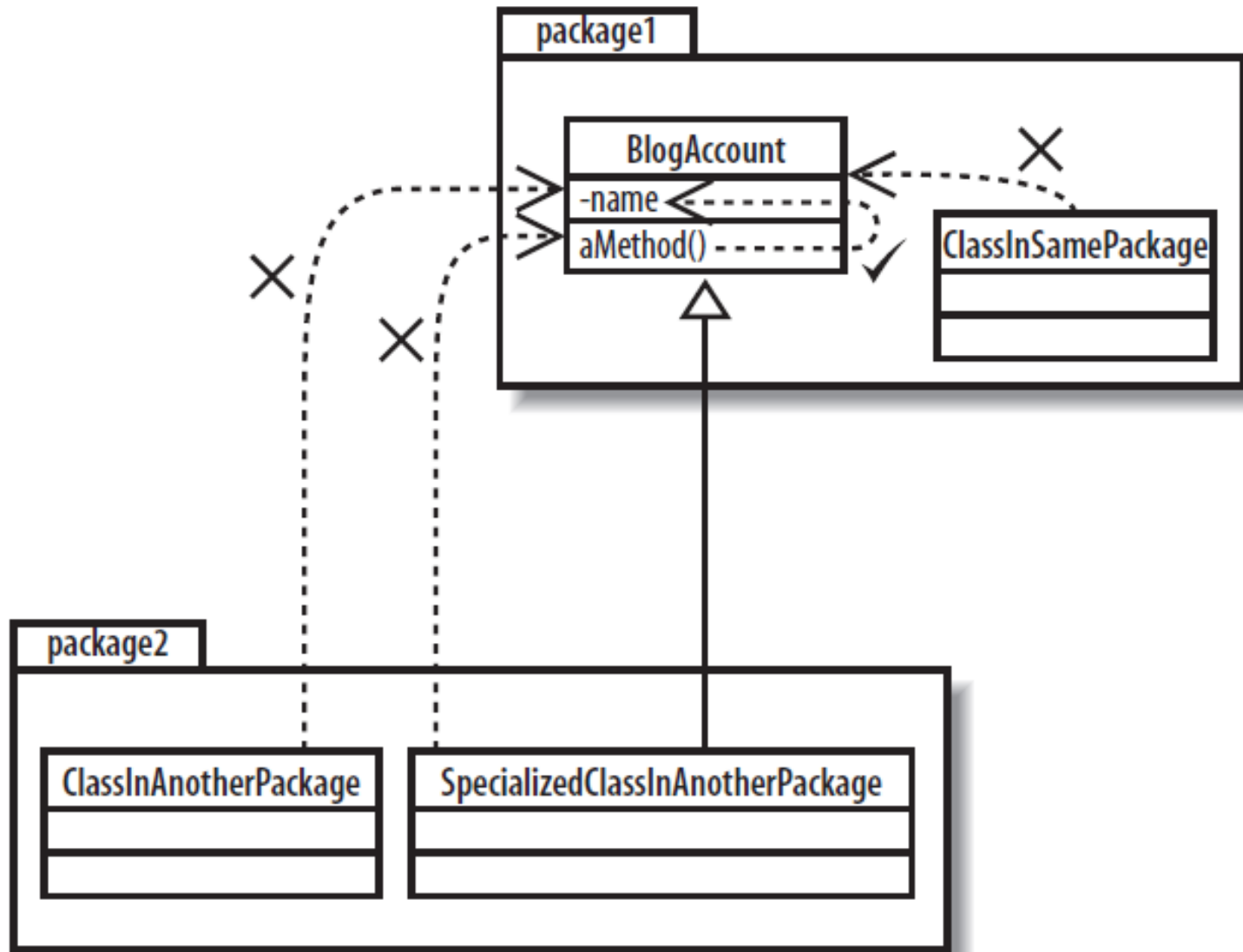
Protected Visibility



Package Visibility



Private Visibility



Data Field Encapsulation

- *Making data fields private protects data and makes the class easy to maintain.*

To prevent direct modifications of data fields, you should declare the data fields private, using the **private** modifier. This is known as *data field encapsulation*.

(Example in NetBeans)

Setter & getter methods

A private data field cannot be accessed by an object from outside the class that defines the private field. However, a client often needs to retrieve and modify a data field. To make a private data field accessible, provide a *getter* method to return its value. To enable a private data field to be updated, provide a *setter* method to set a new value. A getter method is also referred to as an *accessor* and a setter to a *mutator*.

A getter method has the following signature:

public returnType *getPropertyName()*

If the **returnType** is **boolean**, the getter method should be defined as follows by convention:

public boolean *isPropertyName()*

A setter method has the following signature:

public void *setPropertyName(dataType propertyValue)*

Static Class Members

- Every object has its own copy of all the instance variables of the class. In certain cases, only one copy of a particular variable should be *shared* by all objects of a class. A **static field**— called a **class variable**—is used in such cases. A static variable represents **classwide information**— all objects of the class share the *same* piece of data. The declaration of a static variable begins with the keyword `static`.

The static can be:

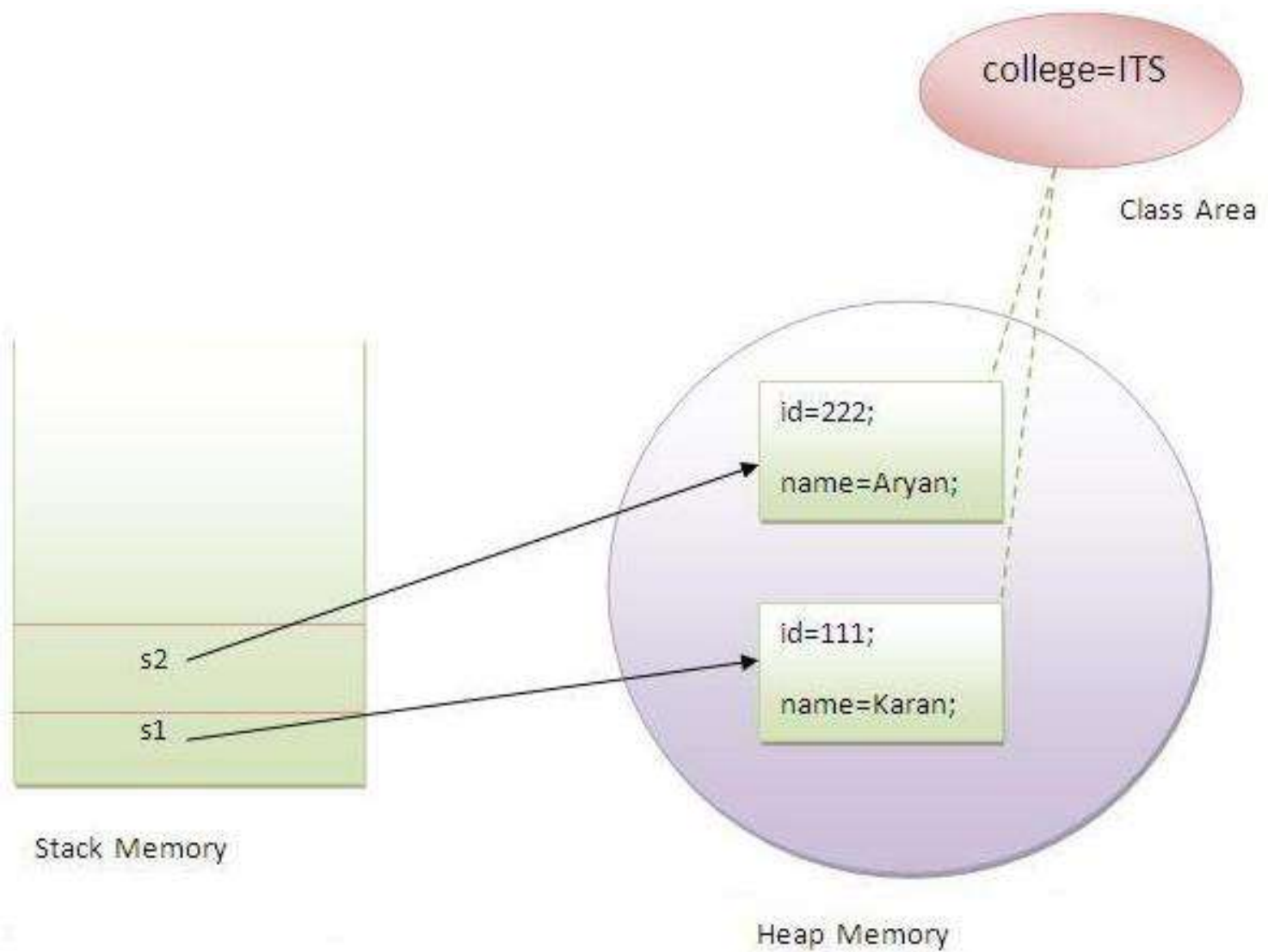
- variable (also known as class variable)
- method (also known as class method)
- block
- nested class

Java static variable

- If you declare any variable as static, it is known static variable.
- The static variable can be used to refer the common property of all objects (that is not unique for each object) e.g. company name of employees, college name of students etc.
- The static variable gets memory only once in class area at the time of class loading.

Advantage of static variable

- It makes your program **memory efficient** (i.e it saves memory).



Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- static method can access static data member and can change the value of it.

Java static block

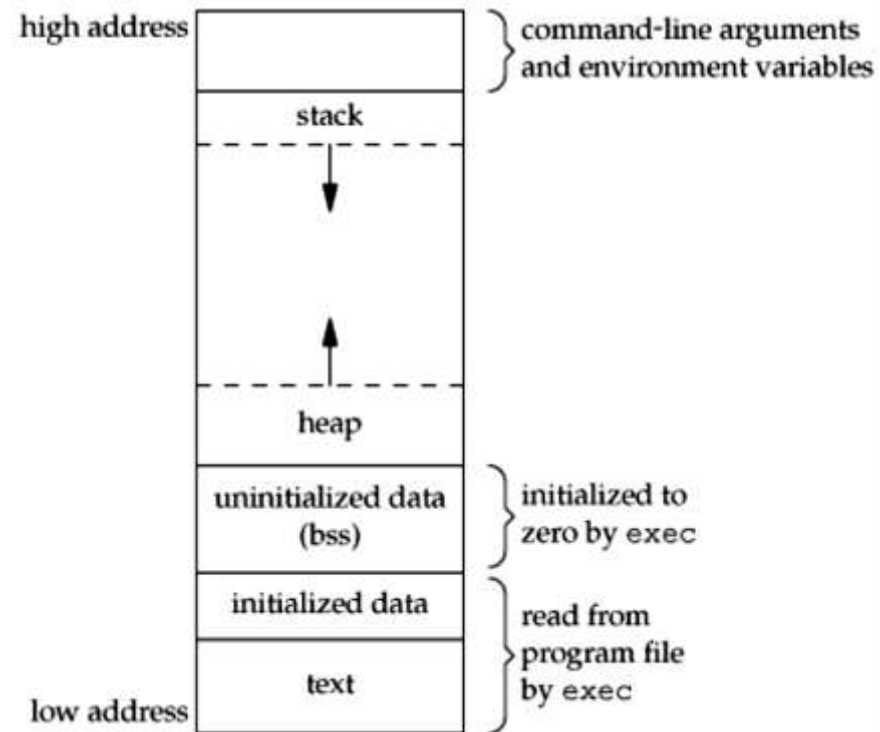
- Is used to initialize the static data member.
- It is executed before main method at the time of class loading.

```
class A2{  
    static{System.out.println("static block is invoked");}  
    public static void main(String args[]){  
        System.out.println("Hello main");  
    }  
}
```

```
Output:static block is invoked  
        Hello main
```

Method-Call Stack and Stack Frames

- To understand how Java performs method calls, we first need to consider a data structure (i.e., collection of related data items) known as a stack. You can think of a stack as analogous to a pile of dishes. When a dish is placed on the pile, it's normally placed at the top (referred to as pushing the dish onto the stack). Similarly, when a dish is removed from the pile, it's normally removed from the top (referred to as popping the dish off the stack).



cont...

Stacks are known as **last-in, first-out (LIFO) data structures**—the *last* item pushed onto the stack is the *first* item popped from the stack.

When a program *calls* a method, the called method must know how to *return* to its caller, so the *return address* of the calling method is *pushed* onto the **method-call stack**. If a series of method calls occurs, the successive return addresses are pushed onto the stack in last-in, first-out order so that each method can return to its caller. The method-call stack also contains the memory for the *local variables* (including the method parameters) used in each invocation of a method during a program's execution. This data, stored as a portion of the method-call stack, is known as the **stack frame** (or **activation record**) of the method call. When a method call is made, the stack frame for that method call is *pushed* onto the method-call stack. When the method returns to its caller, the stack frame for this method call is *popped* off the stack and those local variables are no longer known to the program. If a local variable holding a reference to an object is the only variable in the program with a reference to that object, then, when the stack frame containing that local variable is popped off the stack, the object can no longer be accessed by the program and will eventually be deleted from memory by the JVM during *garbage collection*,

Argument Promotion and Casting

- Another important feature of method calls is **argument promotion**—converting an *argument's value*, if possible, to the type that the method expects to receive in its corresponding *parameter*.

Type	Valid promotions
double	None
float	double
long	float or double
int	long, float or double
char	int, long, float or double
short	int, long, float or double (but not char)
byte	short, int, long, float or double (but not char)
boolean	None (boolean values are not considered to be numbers in Java)

Narrowing Type Casting



Widening Type Casting

Type Casting in Java

Java API Packages

- As you've seen, Java contains many *predefined* classes that are grouped into categories of related classes called *packages*. Together, these are known as the Java Application Programming Interface (Java API), or the Java class library.
- java.awt.event
- java.io
- java.sql
- And many others